

RAG Document Chatbot - Project Documentation

Version 1.0

Date: 23 September 2026

1. Project Overview

1.1 Purpose

The RAG Document Chatbot is a full-stack application that enables conversational question answering over a private set of documents. Users upload files (PDF, DOCX, TXT, Markdown), and the system indexes their content so that subsequent questions are answered strictly from the uploaded material. Each answer is accompanied by the exact source passages used, which makes the system verifiable and suitable for study material, reports, policy documents, and technical manuals.

1.2 What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with large language model generation. Instead of relying on the model's internal knowledge, the system:

1. Converts user documents into a searchable numerical index (vector store).
2. For each question, retrieves the passages most relevant to that question.
3. Instructs the model to compose its answer using only those retrieved passages.

This approach reduces hallucination, works with private data that the model has never seen, and provides provenance through source citations.

1.3 Key Capabilities

- Upload documents through a web interface.
- Automatic text extraction, chunking, and semantic indexing.
- Question answering grounded in uploaded content.
- Source attribution with document name, page number, and snippet.
- Knowledge base reset without redeploying the application.
- Configurable chunking, retrieval depth, embedding model, and chat model.

2. Technology Stack

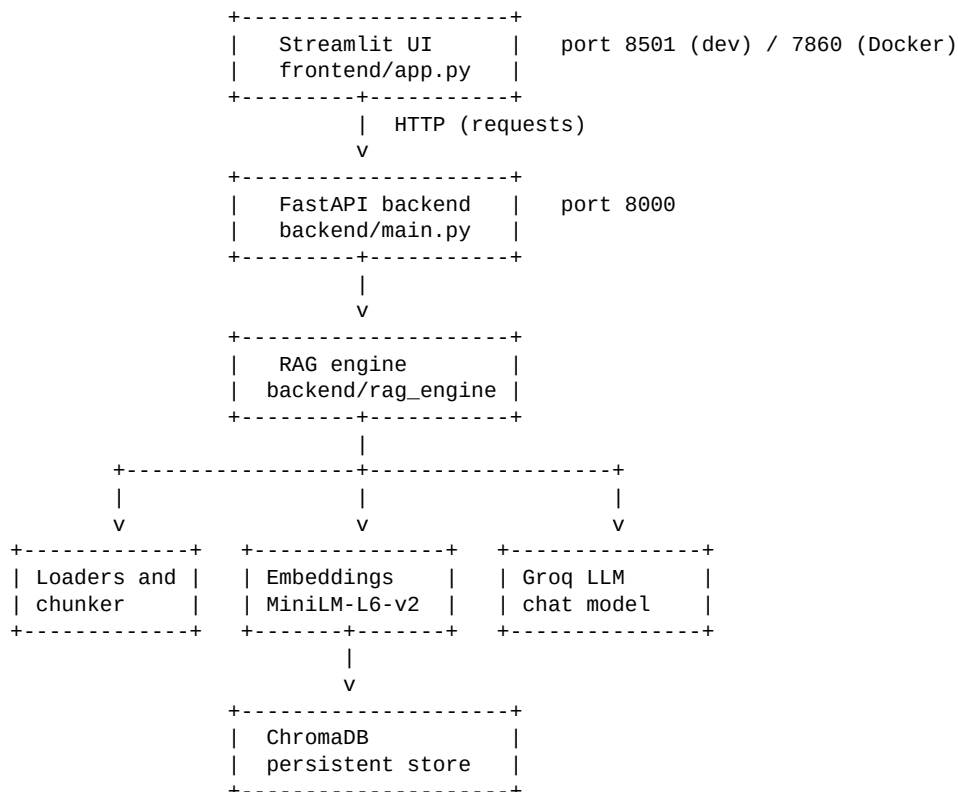
Layer	Technology	Version	Role
Frontend	Streamlit	1.39	Chat interface, file upload, source display
Backend API	FastAPI	0.115	REST endpoints for upload, chat, reset, health
Server	Uvicorn	0.30	ASGI server for FastAPI

RAG framework	LangChain	1.x	Loaders, splitters, integrations
Embeddings	sentence-transformers	5.6	Local embedding of text chunks
Embedding model	all-MiniLM-L6-v2	-	384-dimensional sentence embeddings
Vector store	ChromaDB	0.5	Persistent local similarity search
LLM provider	Groq	-	Hosted inference for the chat model
Document parsing	pypdf, docx2txt	5.1, 0.8	PDF and DOCX text extraction

2.1 Why These Choices

- Local embeddings. all-MiniLM-L6-v2 runs on CPU with no per-request API cost and keeps document content on the machine.
- ChromaDB. Free, embedded, persistent, and requires no external account, which keeps the project fully self-contained.
- Groq. Provides low-latency hosted inference; the model identifier is configuration, not code, so the provider model can be replaced without touching the source.
- FastAPI and Streamlit separation. The API layer is independent of the UI, allowing the frontend to be replaced, the API to be reused by other clients, and each component to scale independently.

3. System Architecture



The frontend and backend communicate over HTTP only. The backend owns all intelligence: document parsing, chunking, embedding, storage, retrieval, and generation. This separation allows the API to be consumed by any client, not just the bundled Streamlit interface.

4. Functional Description

4.1 Document Ingestion Flow

1. The user selects a file in the Streamlit sidebar. Accepted extensions: .pdf, .docx, .txt, .md.
2. The frontend POSTs the file as multipart form data to /upload.
3. The backend validates the extension and writes the file to a temporary location.
4. The appropriate loader parses the document into text pages:
 - PDF: PyPDFLoader (pypdf)
 - DOCX: Docx2txtLoader (docx2txt)
 - TXT/MD: TextLoader with UTF-8 encoding
5. RecursiveCharacterTextSplitter splits the text into chunks of up to CHUNK_SIZE characters with CHUNK_OVERLAP characters of overlap, preferring paragraph, then sentence, then word boundaries.
6. Each chunk is embedded and written to the persistent Chroma collection named rag_documents, with metadata recording the source document name and page number.
7. The API returns the number of pages and chunks created, which the frontend displays.

4.2 Question Answering Flow

1. The user types a question in the chat box.
2. The frontend POSTs {"question", "k"} to /chat.
3. The RAG engine embeds the question and performs similarity search against Chroma, retrieving the k nearest chunks (default 4, adjustable 1-10 via sidebar slider).
4. If no chunks exist, the system answers that the knowledge base is empty.
5. Retrieved chunks are formatted into a prompt that includes the source name of each passage and instructs the model to answer only from the context, to be concise, and to admit when the answer is absent.
6. The prompt is sent to the Groq chat model with temperature 0 for deterministic behavior.
7. The answer and the source list (document, page, 200-character snippet) are returned and rendered.

4.3 Knowledge Base Reset

The /reset endpoint deletes the Chroma collection. The in-memory vector store handle is discarded so the next request recreates a fresh, empty collection. The frontend clears its session document list.

5. API Reference

Base URL: <http://localhost:8000> (development) or the deployed service URL.

Method	Path	Body	Response	Notes
GET	<code>/health`</code>	-	<code>{"status": "ok"}`</code>	Liveness check
POST	<code>/upload`</code>	multipart <code>file`</code>	<code>{"status", "source", "num_pages", "num_chunks"}`</code>	Max file size governed by server defaults
POST	<code>/chat`</code>	<code>{"question": string, "k": integer}`</code>	<code>{"answer": string, "sources": [{"source", "page", "snippet"}]}`</code>	<code>k`</code> defaults to 4
POST	<code>/reset`</code>	-	<code>{"status": "knowledge base cleared"}`</code>	Irreversible
GET	<code>/docs`</code>	-	Swagger UI	Auto-generated by FastAPI

Error handling: invalid file types return HTTP 400; empty questions return HTTP 400; processing failures return HTTP 500 with the underlying error message.

6. Project Structure

```

rag-chatbot/
├── backend/
│   ├── main.py           FastAPI application and endpoint definitions
│   └── rag_engine.py     Core RAG logic (loading, chunking, embedding, QA)
├── frontend/
│   └── app.py           Streamlit chat interface
├── tests/
│   └── smoke_document.txt Sample text used for end-to-end smoke testing
├── requirements.txt      Pinned dependencies for both services
├── Dockerfile           Single-image build for combined deployment
├── start.sh             Container entrypoint script
├── .env.example         Configuration template (no secrets)
├── .gitignore           Excludes secrets, caches, and environments
├── README.md           Operational readme
├── docs/
│   ├── PROJECT_DOCUMENTATION.md This document (Markdown source)
│   └── PROJECT_DOCUMENTATION.pdf Printable PDF edition

```

7. Configuration

All runtime parameters are environment variables with sensible defaults, loaded from the `.env` file.

Variable	Default	Purpose
<code>GROQ_API_KEY`</code>	none, required	Authentication for Groq inference
<code>CHAT_MODEL`</code>	<code>openai/gpt-oss-120b`</code>	Groq chat model identifier
<code>EMBEDDING_MODEL`</code>	<code>sentence-transformers/all-MiniLM-L6-v2`</code>	Embedding model for chunks and queries
<code>CHUNK_SIZE`</code>	<code>1000`</code>	Maximum characters per chunk
<code>CHUNK_OVERLAP`</code>	<code>150`</code>	Character overlap between consecutive chunks
<code>CHROMA_DIR`</code>	<code>chroma_db`</code>	Filesystem path of the persistent vector store
<code>BACKEND_URL`</code>	<code>http://localhost:8000`</code>	API base URL used by the frontend
<code>HF_HOME`</code>	<code>.hf-cache`</code>	Hugging Face download cache location

Security rule: the real .env file containing GROQ_API_KEY is excluded by .gitignore and must never be committed. On hosted platforms, the key is provided through the platform's secret manager.

8. Running the Project

8.1 Local Development

Prerequisites: Python 3.10+ and a Groq API key.

```
# Terminal 1 - backend
cd rag-chatbot
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env          # then set GROQ_API_KEY inside
cd backend
uvicorn main:app --port 8000

# Terminal 2 - frontend
cd rag-chatbot\frontend
..\venv\Scripts\Activate.ps1
streamlit run app.py
```

Access points:

- Web interface: <http://localhost:8501>
- API health: <http://localhost:8000/health>
- Swagger documentation: <http://localhost:8000/docs>

8.2 Docker

```
docker build -t rag-chatbot .
docker run -p 7860:7860 --env-file .env rag-chatbot
```

The container starts Uvicorn on internal port 8000, then Streamlit on port 7860, which is the externally exposed port. This matches the Hugging Face Spaces convention.

8.3 Smoke Test

A minimal end-to-end verification:

```
# with the backend running
curl -F "file=@tests/smoke_document.txt" http://localhost:8000/upload
curl -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"question": "What is"}'
```

Expected: the upload reports chunk counts, and the chat returns an answer with source metadata.

9. Deployment

9.1 Hugging Face Spaces (recommended free option)

1. Create a Space at <https://huggingface.co/new-space> with SDK set to Docker.

2. Push the repository contents to the Space repository. Required files: Dockerfile, start.sh, requirements.txt, backend/, frontend/.
3. Add GROQ_API_KEY in Space settings under Variables and secrets.
4. The Space builds the image and serves the application on port 7860.

Limitation: the Space filesystem is ephemeral. On restart, the ChromaDB data is cleared and documents must be re-uploaded.

9.2 Render or Railway

1. Create a Web Service from the repository.
2. Choose the Docker runtime; the existing Dockerfile is used directly.
3. Set the GROQ_API_KEY environment variable in the dashboard.
4. Publish port 7860.

9.3 Virtual Private Server

```
git clone <repository-url>
cd rag-chatbot
cp .env.example .env && nano .env
docker build -t rag-chatbot .
docker run -d -p 80:7860 --env-file .env --restart unless-stopped rag-chatbot
```

Terminate TLS in front of the container (for example with nginx or a cloud load balancer) for production traffic.

10. Design Decisions and Limitations

10.1 Design Decisions

- Model as configuration. The chat model identifier lives in CHAT_MODEL, never in code. Provider model catalogs change; configuration-only changes absorb those changes.
- Answer grounding. The prompt template forbids answering outside the retrieved context and prefers an explicit "not in the documents" response over speculation.
- Temperature 0. Deterministic output improves reproducibility for a question-answering product.
- Persistent local vector store. ChromaDB writes to disk, so documents survive backend restarts in development.
- Stateless API. All state lives in the vector store and the client session, which keeps the API horizontally scalable.

10.2 Known Limitations

- Shared knowledge base. A single collection serves all users. Multi-tenant isolation requires collection namespacing by user or session.
- Ephemeral container storage. Vector data inside a container is lost on restart unless a volume is mounted or a managed vector database is used.
- Scanned PDFs. Image-only PDFs require OCR before ingestion; no OCR step is bundled.

- English-centric embeddings. all-MiniLM-L6-v2 is strongest on English text; multilingual corpora would benefit from a multilingual embedding model.
- No authentication. The API and UI are unauthenticated by design for a demo; production use requires adding identity and access control.

11. Testing

- Manual smoke test. Upload tests/smoke_document.txt, confirm chunk counts, then ask a question whose answer is present in the document and verify sources are returned.
- API validation. Exercise /health, /docs, invalid upload types (expect 400), and empty chat questions (expect 400).
- Reset verification. After /reset, the chat endpoint should report an empty knowledge base.

12. Future Enhancements

1. Per-user knowledge bases through collection namespacing.
2. Conversation memory so follow-up questions resolve pronouns and context.
3. Streaming responses for lower perceived latency.
4. OCR pipeline for scanned documents.
5. Persistent cloud vector store option (Pinecone or equivalent) for production deployments.
6. Evaluation harness measuring retrieval precision and answer faithfulness.

13. License and Credits

Built with LangChain, ChromaDB, sentence-transformers, FastAPI, Streamlit, and Groq. Embedding model: sentence-transformers/all-MiniLM-L6-v2 by the sentence-transformers project.